

# RAPPORT DE PROJET BACK-END

## Kanban Board API

Développement d'une API REST avec NestJS

**Étudiant**

Fallou Gaye

**Formation**

Master Développement & DevOps

Année 2026

# Table des matières

|           |                                            |           |
|-----------|--------------------------------------------|-----------|
| <b>1</b>  | <b>Introduction</b>                        | <b>2</b>  |
| <b>2</b>  | <b>Organisation du projet</b>              | <b>2</b>  |
| <b>3</b>  | <b>Base de données et modélisation</b>     | <b>2</b>  |
| <b>4</b>  | <b>Documentation et tests avec Swagger</b> | <b>3</b>  |
| <b>5</b>  | <b>Inscription d'un utilisateur</b>        | <b>4</b>  |
| <b>6</b>  | <b>Connexion et authentification</b>       | <b>6</b>  |
| <b>7</b>  | <b>Récupération du profil utilisateur</b>  | <b>7</b>  |
| <b>8</b>  | <b>Gestion des listes</b>                  | <b>8</b>  |
| 8.1       | Création d'une liste . . . . .             | 8         |
| 8.2       | Récupération des listes . . . . .          | 9         |
| <b>9</b>  | <b>Gestion des cartes</b>                  | <b>10</b> |
| 9.1       | Création d'une carte . . . . .             | 10        |
| 9.2       | Récupération des cartes . . . . .          | 11        |
| 9.3       | Modification d'une carte . . . . .         | 12        |
| <b>10</b> | <b>Sécurité</b>                            | <b>13</b> |
| <b>11</b> | <b>Tests automatisés</b>                   | <b>14</b> |
| <b>12</b> | <b>Difficultés rencontrées</b>             | <b>15</b> |
| <b>13</b> | <b>Conclusion</b>                          | <b>16</b> |

# 1. Introduction

Dans le cadre de ce projet, j'ai développé le back-end d'une application de type Kanban Board. Le but était de mettre en place une API permettant à un utilisateur de créer un compte, de se connecter et de gérer ses propres listes et cartes.

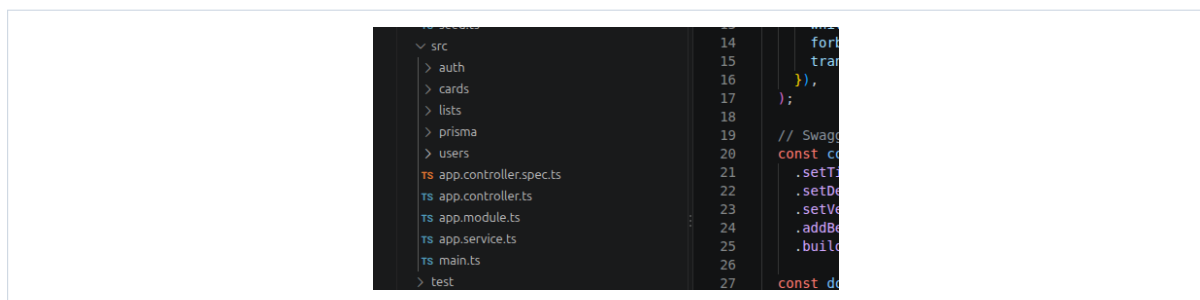
J'ai utilisé NestJS avec TypeScript pour développer l'API. Pour la partie base de données, j'ai travaillé avec PostgreSQL et Prisma. J'ai également intégré Swagger afin de documenter les routes et surtout de pouvoir les tester directement pendant le développement.

Le projet est organisé autour de plusieurs parties : l'authentification, les utilisateurs, les listes, les cartes et l'accès à la base de données. Dans ce rapport, je présente l'organisation du back-end ainsi que les principales fonctionnalités que j'ai développées et testées.

# 2. Organisation du projet

J'ai organisé le projet en plusieurs modules afin de séparer les différentes fonctionnalités. Cette organisation m'a permis de mieux comprendre le rôle de chaque partie de l'application et d'éviter de regrouper toute la logique dans les mêmes fichiers.

Dans le dossier `src`, on retrouve principalement les modules `auth`, `users`, `lists`, `cards` et `prisma`. Le module `auth` s'occupe de l'inscription et de la connexion, tandis que le module `users` gère les utilisateurs. Les modules `lists` et `cards` correspondent aux deux éléments principaux du Kanban. Enfin, la partie `prisma` permet au back-end de communiquer avec PostgreSQL.

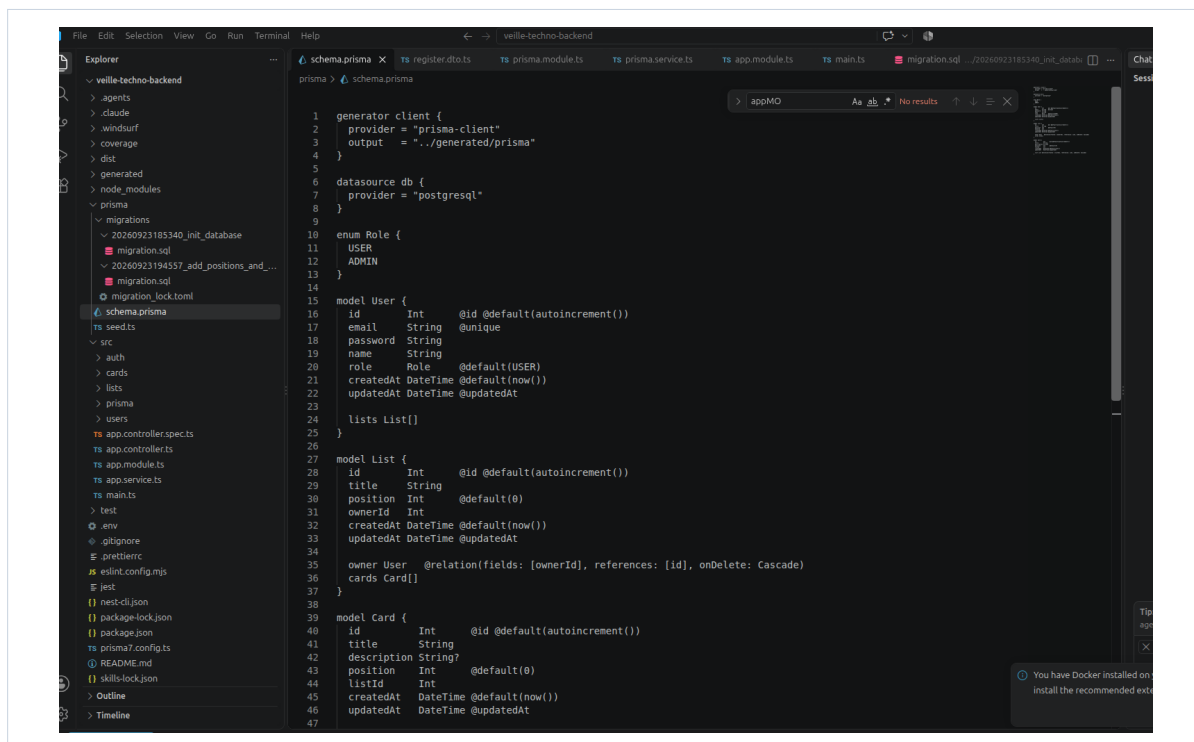


**FIGURE 1** – Organisation des principaux modules du projet

Dans NestJS, le contrôleur reçoit la requête HTTP et appelle ensuite le service correspondant. Le service contient la logique nécessaire au traitement de la demande. Lorsque des données doivent être enregistrées ou récupérées, Prisma communique avec PostgreSQL.

# 3. Base de données et modélisation

J'ai utilisé PostgreSQL pour stocker les données de l'application. La structure des données est définie avec Prisma. Le projet contient trois modèles principaux : `User`, `List` et `Card`.



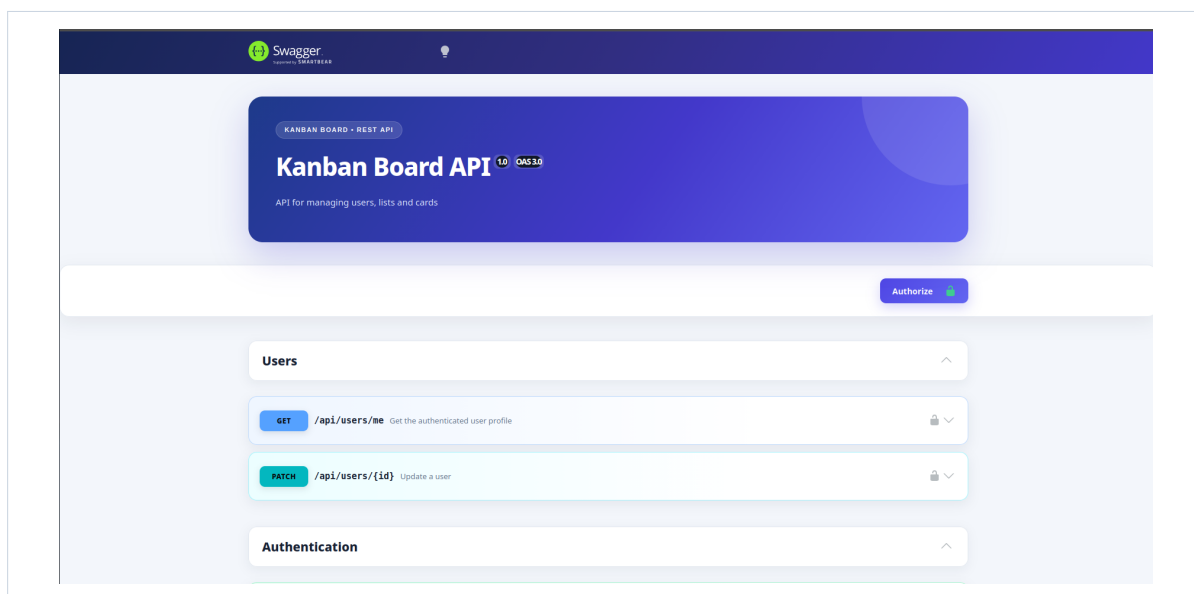
**FIGURE 2** – Modélisation des utilisateurs, listes et cartes avec Prisma

Le modèle User contient notamment l'identifiant, l'adresse e-mail, le mot de passe, le nom et le rôle de l'utilisateur. Une List représente une liste du Kanban. Elle contient un titre, une position et un ownerId. Ce dernier permet de relier la liste à l'utilisateur qui l'a créée.

Une Card représente une tâche. Elle contient notamment un titre, une description, une position et un listId. Le listId permet de relier chaque carte à sa liste. La structure obtenue suit donc la relation User -> List -> Card.

## 4. Documentation et tests avec Swagger

J'ai utilisé Swagger pour documenter les routes de l'API. La documentation est accessible sur la route /api.

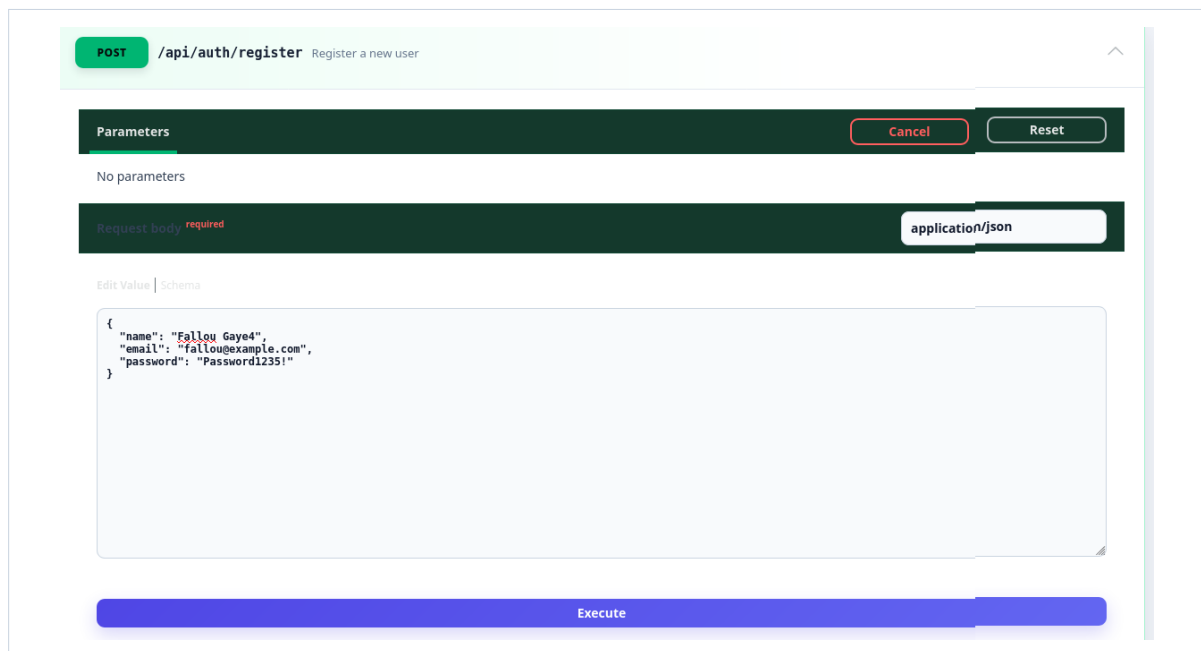


**FIGURE 3** – Interface Swagger de la Kanban Board API

Swagger regroupe les routes par fonctionnalité. On retrouve les utilisateurs, l’authentification, les listes et les cartes. Je l’ai également utilisé comme outil de test pendant le développement. Pour chaque fonctionnalité, j’ai envoyé une requête depuis Swagger puis vérifié le code HTTP et les données retournées par le serveur. Cela m’a permis de tester progressivement l’API sans avoir besoin de développer une interface front-end.

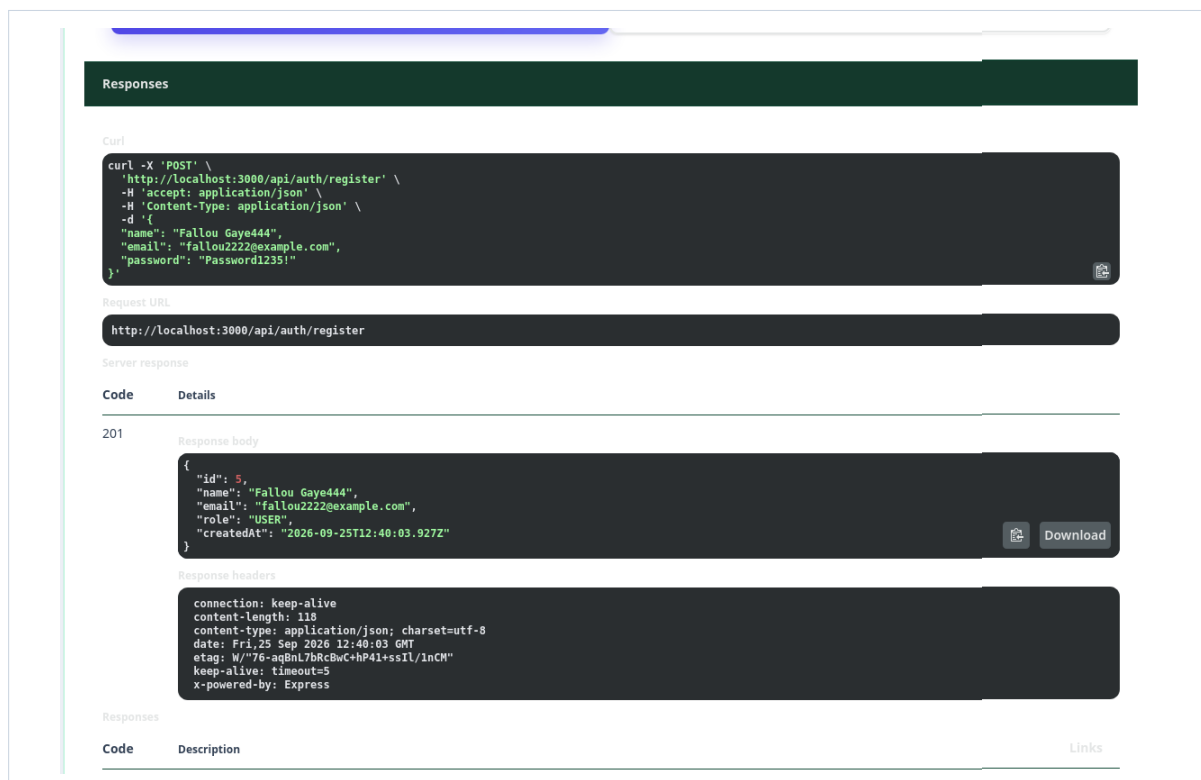
## 5. Inscription d’un utilisateur

La première étape consiste à permettre la création d’un compte. Pour cela, j’ai développé la route POST `/api/auth/register`. Pour tester cette route, j’ai envoyé un nom, une adresse e-mail et un mot de passe depuis Swagger.



**FIGURE 4** – Test de la route d’inscription

Le but de ce test était de vérifier que les informations envoyées étaient bien acceptées par l’API et qu’un nouvel utilisateur pouvait être créé.



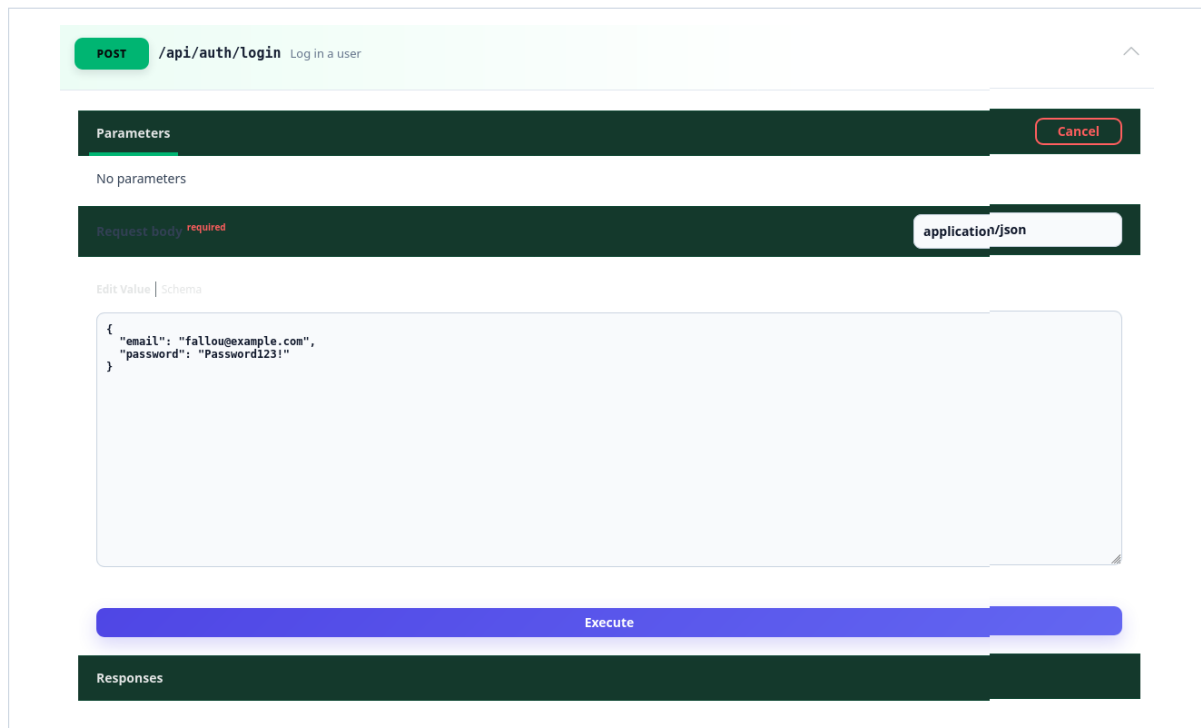
**FIGURE 5** – Résultat de la création d’un utilisateur

Le serveur retourne le code HTTP **201**, ce qui confirme que la ressource a bien été créée. La réponse contient l’identifiant de l’utilisateur, son nom, son adresse e-mail, son rôle et sa date de

création. Le mot de passe n'apparaît pas dans la réponse. Lors de l'inscription, le compte reçoit le rôle USER par défaut.

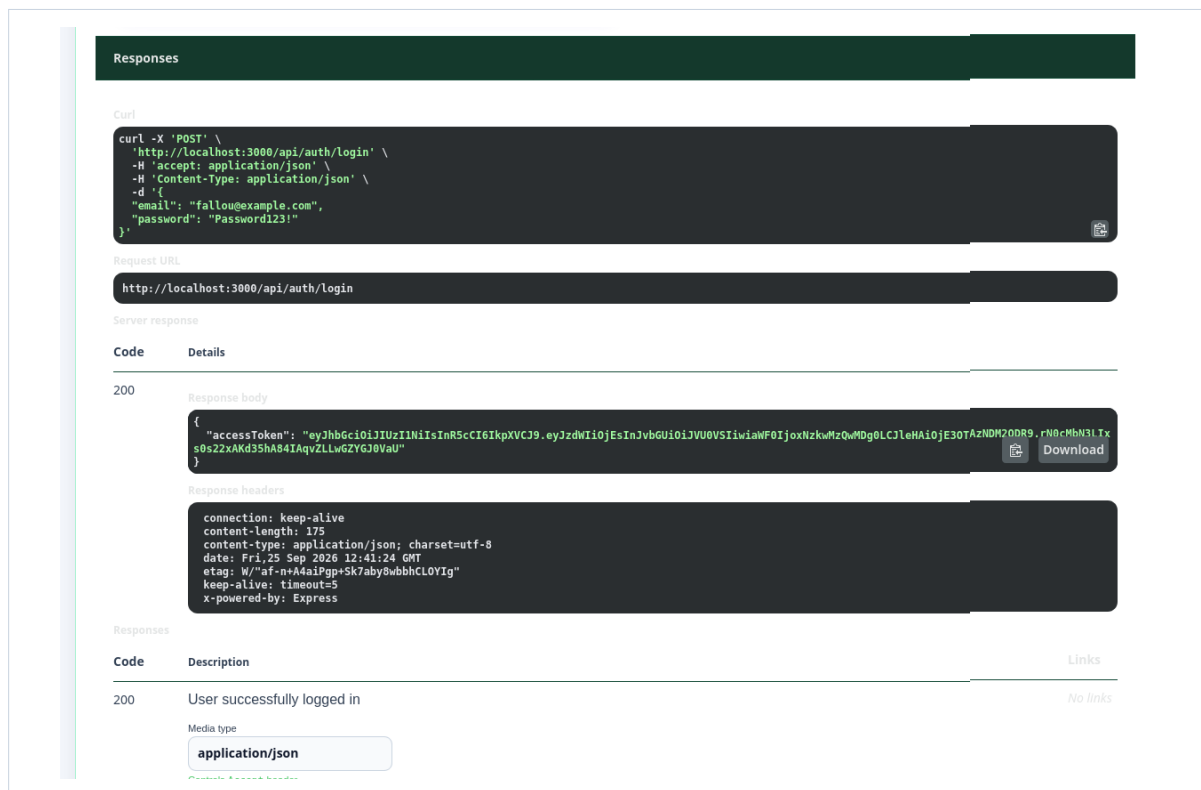
## 6. Connexion et authentification

Après l'inscription, j'ai testé la connexion avec la route POST `/api/auth/login`. Le test consiste à envoyer l'adresse e-mail et le mot de passe d'un utilisateur déjà enregistré.



**FIGURE 6** – Test de la connexion d'un utilisateur

L'objectif était de vérifier que l'API reconnaissait correctement les informations de connexion et générait un token lorsque l'utilisateur était authentifié.



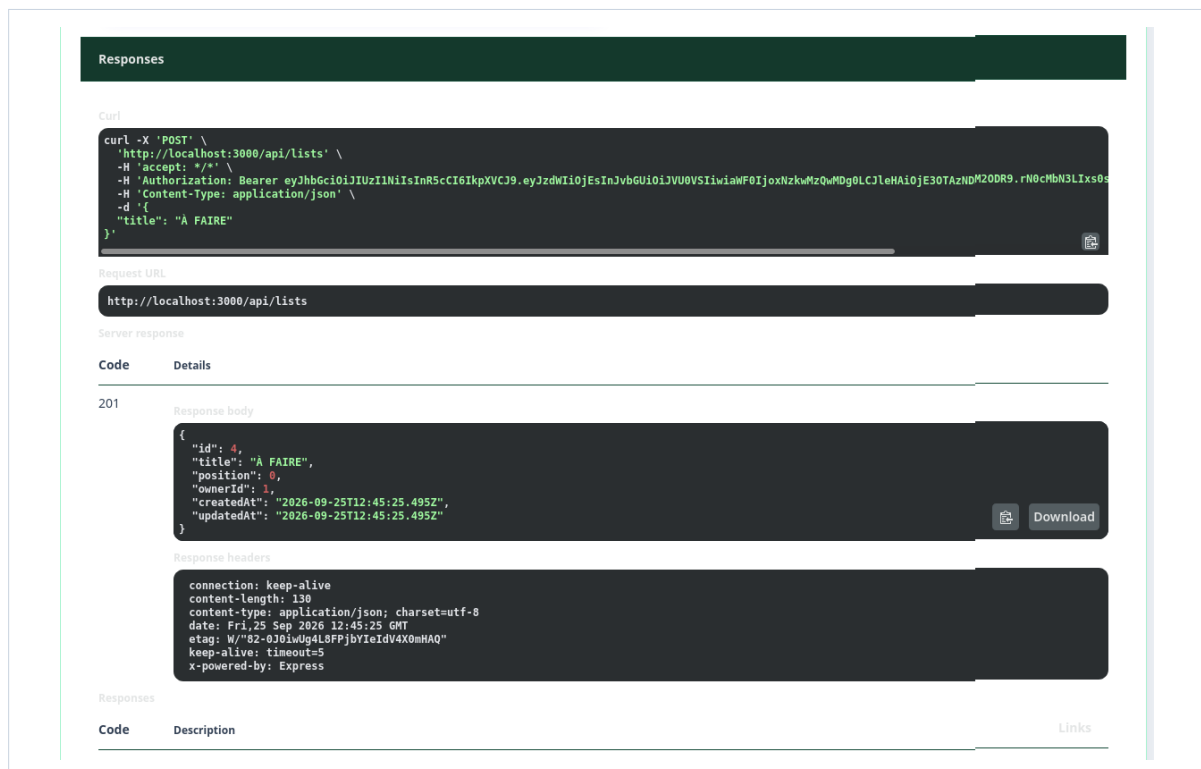
**FIGURE 7** – Résultat de l'authentification et génération du JWT

La requête retourne le code HTTP **200** ainsi qu'un accessToken. Ce token JWT est ensuite utilisé pour accéder aux routes protégées. Dans Swagger, je l'ai renseigné avec le bouton *Authorize*. Il est alors envoyé avec les requêtes qui nécessitent une authentification.

## 7. Récupération du profil utilisateur

Après la connexion, j'ai testé la route GET `/api/users/me`. Cette route permet de récupérer le profil correspondant à l'utilisateur actuellement connecté.





**FIGURE 9** – Création de la liste « À FAIRE »

Le but de ce test était de vérifier qu'un utilisateur authentifié pouvait créer une liste et que cette liste lui était correctement associée. Le serveur retourne le code HTTP **201**. La réponse contient notamment l'identifiant de la liste, son titre, sa position et son `ownerId`. Dans le résultat obtenu, `ownerId` vaut 1. La liste est donc bien associée à l'utilisateur qui l'a créée.

## 8.2. Récupération des listes

J'ai ensuite testé la récupération des listes avec la route `GET /api/lists`.

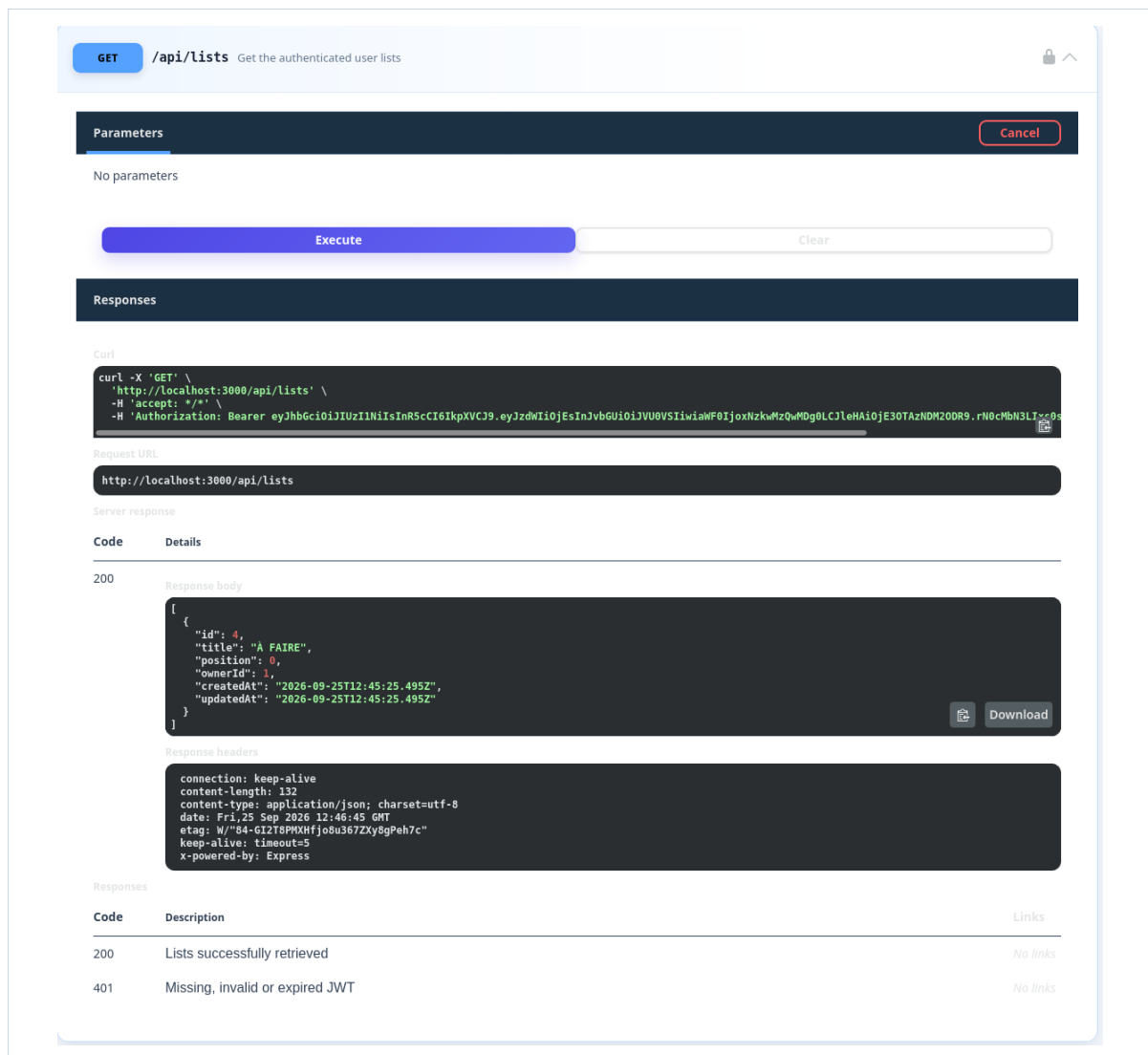


FIGURE 10 – Récupération des listes de l'utilisateur

L'objectif de ce test était de vérifier que l'utilisateur connecté pouvait retrouver les listes qui lui appartiennent. Le serveur retourne le code HTTP **200** et la liste « **À FAIRE** » précédemment créée.

## 9. Gestion des cartes

### 9.1. Création d'une carte

Après la création de la liste, j'ai testé l'ajout d'une carte avec la route POST `/api/lists/{listId}/cards`. Dans ce test, j'ai utilisé la liste ayant l'identifiant 4.

**POST** /api/lists/{listId}/cards Create a card in a list

**Parameters** Cancel Reset

| Name                                             | Description |
|--------------------------------------------------|-------------|
| listId <span>required</span><br>number<br>(path) | 4           |

**Request body** required application/json

Edit Value | Schema

```
{
  "title": "À FAIRE",
  "description": "Mettre en place JWT et les Guards"
}
```

Execute Clear

**Responses**

curl -X 'POST' \
 'http://localhost:3000/api/lists/4/cards' \
 -H 'accept: \*/\*' \
 -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiEsInJvbGU1OjVUUVV5IiwiaWF0IjoxNzkwMzQwMDg0LCJleHAiOiE3OTAzNDMzODR9.fN0cHbN3Lixs0s' \
 -H 'Content-Type: application/json' \
 -d '{
 "title": "À FAIRE",
 "description": "Mettre en place JWT et les Guards"
 }'

Request URL  
http://localhost:3000/api/lists/4/cards

Server response

| Code | Details                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 201  | <p><b>Response body</b></p> <pre>{   "id": 3,   "title": "À FAIRE",   "description": "Mettre en place JWT et les Guards",   "position": 0,   "listId": 4,   "createdAt": "2026-09-25T12:51:58.811Z",   "updatedAt": "2026-09-25T12:51:58.811Z" }</pre> <p><span>Download</span></p> <p><b>Response headers</b></p> <pre>connection: keep-alive content-length: 179 content-type: application/json; charset=utf-8 date: Fri, 25 Sep 2026 12:51:58 GMT etag: W/"b3-e7ZdcZtyKehv8jhBvqjL6rKvqyo" keep-alive: timeout=5 x-powered-by: Express</pre> |

**Responses**

| Code | Description |
|------|-------------|
|------|-------------|

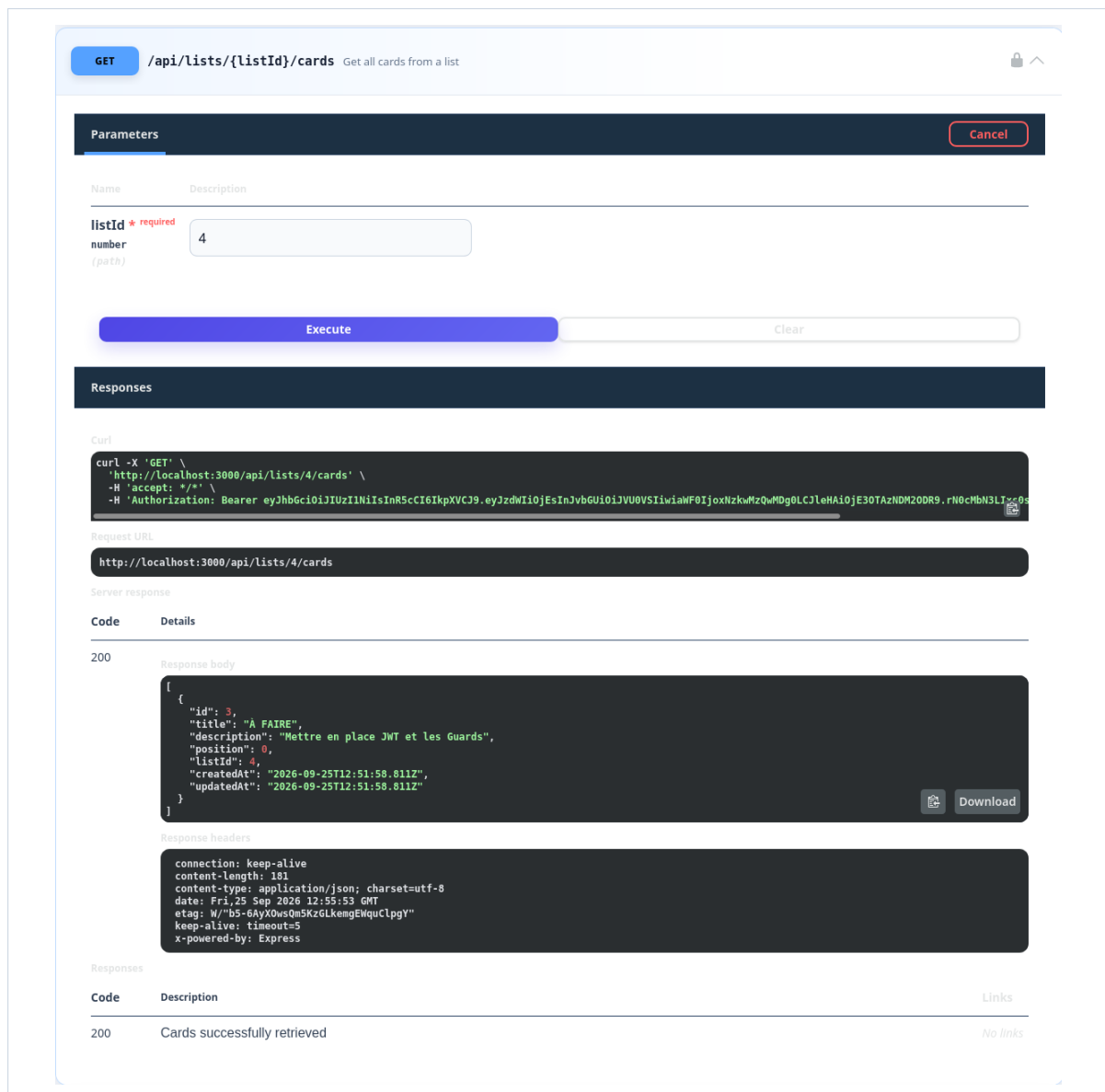
[Links](#)

**FIGURE 11** – Création d’une carte dans la liste numéro 4

Le but était de vérifier qu’une carte pouvait être créée dans une liste existante et que la relation entre la carte et la liste était correctement enregistrée. Le serveur retourne le code HTTP **201**. La carte créée possède l’identifiant 3 et son champ `listId` vaut 4. Cela confirme que la carte a bien été rattachée à la liste utilisée pendant le test.

## 9.2. Récupération des cartes

J’ai ensuite utilisé la route GET `/api/lists/{listId}/cards` pour récupérer les cartes de la liste numéro 4.

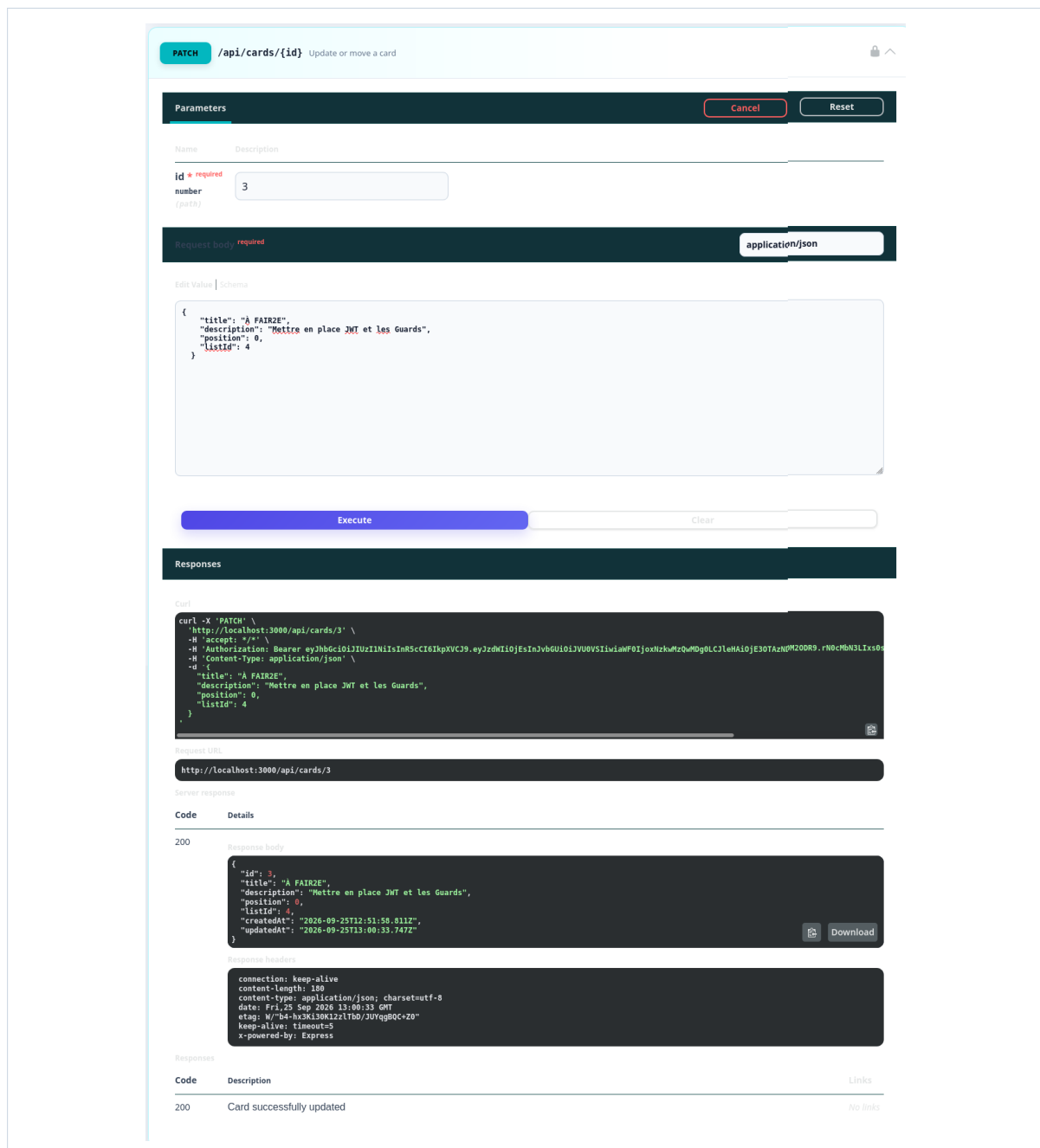


**FIGURE 12** – Récupération des cartes de la liste numéro 4

Le but de ce test était de vérifier que les cartes enregistrées dans une liste pouvaient être récupérées correctement. La réponse retourne le code HTTP **200** ainsi qu'un tableau contenant la carte créée précédemment. On retrouve notamment son identifiant, son titre, sa description et son `listId`.

### 9.3. Modification d'une carte

Enfin, j'ai testé la modification d'une carte avec la route `PATCH /api/cards/{id}`. Pour ce test, j'ai utilisé la carte ayant l'identifiant 3.



**FIGURE 13** – Modification de la carte numéro 3

L'objectif était de vérifier qu'une carte existante pouvait être mise à jour sans avoir à la recréer. La requête retourne le code HTTP **200**. Les nouvelles informations sont directement visibles dans la réponse. La valeur `updatedAt` est également différente de `createdAt`, ce qui permet de constater que la modification a bien été enregistrée.

## 10. Sécurité

La sécurité a été prise en compte dès la partie authentification. Les mots de passe sont hachés avant leur enregistrement dans la base de données et ne sont pas retournés dans les réponses de l'API. Après la connexion, l'utilisateur reçoit un token JWT qui permet de l'identifier lorsqu'il

utilise une route protégée.

J'ai utilisé un Guard NestJS pour vérifier la présence et la validité du JWT avant d'autoriser l'accès à ces routes. Les listes sont également liées à leur propriétaire avec `ownerId`, tandis que les cartes sont reliées à leur liste avec `listId`. Cette organisation permet de contrôler l'utilisateur qui accède aux différentes ressources. Les DTO et le système de validation de NestJS permettent également de contrôler les données envoyées à l'API avant leur traitement.

## 11. Tests automatisés

En complément des tests effectués avec Swagger, j'ai utilisé Jest pour automatiser les tests du back-end. Ces tests permettent de vérifier le fonctionnement des principales parties de l'application, notamment l'authentification, les utilisateurs, les listes, les cartes, le contrôle du JWT et l'accès aux données avec Prisma. L'intérêt est de pouvoir relancer rapidement l'ensemble des tests après une modification du code et de vérifier que les fonctionnalités déjà développées continuent de fonctionner correctement.

Pour vérifier la couverture du projet, j'ai exécuté la commande `npm run test:cov`. Au total, **11 suites de tests** ont été exécutées avec succès et les **75 tests** présents dans le projet ont été validés, sans aucun échec.

```

fallou@fallougaye:~/Bureau/Master1 devop/veille-techno-backend$ ^C
fallou@fallougaye:~/Bureau/Master1 devop/veille-techno-backend$ ^C
fallou@fallougaye:~/Bureau/Master1 devop/veille-techno-backend$ npm run test:cov

> veille-techno-backend@0.0.1 test:cov
> jest --coverage

PASS src/auth/auth.service.spec.ts (6.245 s)
PASS src/cards/cards.service.spec.ts (6.39 s)
PASS src/users/users.service.spec.ts (6.693 s)
PASS src/lists/lists.controller.spec.ts
PASS src/cards/cards.controller.spec.ts
PASS src/users/users.controller.spec.ts
PASS src/lists/lists.service.spec.ts
PASS src/auth/auth.controller.spec.ts
PASS src/app.controller.spec.ts
PASS src/auth/guards/jwt-auth.guard.spec.ts
PASS src/prisma/prisma.service.spec.ts

File | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-----|-----|-----|-----|-----|-----
All files | 82.18 | 95.94 | 95.45 | 84.15 |
src | 38.23 | 0 | 75 | 32.14 |
  app.controller.ts | 100 | 100 | 100 | 100 |
  app.module.ts | 0 | 100 | 100 | 0 | 1-22
  app.service.ts | 100 | 100 | 100 | 100 |
  main.ts | 0 | 0 | 0 | 0 | 1-653
  src/auth | 79.16 | 100 | 85.71 | 80.95 |
    auth.controller.ts | 100 | 100 | 100 | 100 |
    auth.module.ts | 0 | 100 | 0 | 0 | 1-26
    auth.service.ts | 100 | 100 | 100 | 100 |
  src/auth/dto | 100 | 100 | 100 | 100 |
    login.dto.ts | 100 | 100 | 100 | 100 |
    register.dto.ts | 100 | 100 | 100 | 100 |
  src/auth/guards | 100 | 100 | 100 | 100 |
    jwt-auth.guard.ts | 100 | 100 | 100 | 100 |
  src/cards | 89.85 | 100 | 100 | 92.06 |
    cards.controller.ts | 100 | 100 | 100 | 100 |
    cards.module.ts | 0 | 100 | 100 | 0 | 1-12
    cards.service.ts | 100 | 100 | 100 | 100 |
  src/cards/dto | 100 | 100 | 100 | 100 |
    create-card.dto.ts | 100 | 100 | 100 | 100 |
    update-card.dto.ts | 100 | 100 | 100 | 100 |
  src/lists | 84.44 | 100 | 100 | 87.17 |
    lists.controller.ts | 100 | 100 | 100 | 100 |
    lists.module.ts | 0 | 100 | 100 | 0 | 1-11
    lists.service.ts | 100 | 100 | 100 | 100 |
  src/lists/dto | 100 | 100 | 100 | 100 |
    create-list.dto.ts | 100 | 100 | 100 | 100 |
    update-list.dto.ts | 100 | 100 | 100 | 100 |
  src/prisma | 70.58 | 100 | 100 | 76.92 |
    prisma.module.ts | 0 | 100 | 100 | 0 | 1-9
    prisma.service.ts | 100 | 100 | 100 | 100 |
  src/users | 85.41 | 95.45 | 100 | 88.09 |
    users.controller.ts | 100 | 100 | 100 | 100 |
    users.module.ts | 0 | 100 | 100 | 0 | 1-12
    users.service.ts | 100 | 95.45 | 100 | 100 | 78
  src/users/dto | 100 | 100 | 100 | 100 |
    update-user.dto.ts | 100 | 100 | 100 | 100 |

Test Suites: 11 passed, 11 total
Tests: 75 passed, 75 total
Snapshots: 0 total
Time: 15.64 s
Ran all test suites.
fallou@fallougaye:~/Bureau/Master1 devop/veille-techno-backend$

```

FIGURE 14 – Résultat des tests automatisés et couverture du code

La couverture obtenue est de **82,18 %** pour les instructions et de **84,15 %** pour les lignes de code. La couverture des fonctions atteint **95,45 %** et celle des branches **95,94 %**. Ces résultats montrent que les tests automatisés couvrent une partie importante du back-end et complètent les tests fonctionnels réalisés avec Swagger.

## 12. Difficultés rencontrées

Pendant le développement, la configuration de PostgreSQL et de Prisma m'a demandé plusieurs ajustements. J'ai notamment dû comprendre le fonctionnement du schéma Prisma, des migrations et de la connexion entre l'application et PostgreSQL.

La partie authentification m'a également demandé de bien comprendre le fonctionnement d'un JWT et la manière de protéger une route avec un Guard. J'ai ensuite intégré Swagger pour avoir une vue claire des routes et pour pouvoir tester chaque fonctionnalité au fur et à mesure du développement.

Ces différentes étapes m'ont permis de mieux comprendre le rôle des contrôleurs, des services et de Prisma dans une application NestJS.

## 13. Conclusion

Ce projet m'a permis de développer le back-end d'une application Kanban avec NestJS. L'API permet de créer un utilisateur, de se connecter, de récupérer son profil, de créer et récupérer des listes, puis de créer, consulter et modifier les cartes associées à ces listes.

J'ai utilisé PostgreSQL pour stocker les données, Prisma pour communiquer avec la base et Swagger pour documenter et tester les routes. L'authentification repose sur JWT.

Au-delà des fonctionnalités réalisées, ce projet m'a surtout permis de mieux comprendre l'organisation d'un projet NestJS et la manière dont les différentes parties d'un back-end communiquent entre elles.